

DC-OPS

On-Device Data Center Operations Assistant

Qualcomm x Meta ExecuTorch Hackathon — June 2026

Real-time AI-powered server rack inspection

Running entirely on Samsung Galaxy S25 Ultra — Snapdragon 8 Elite NPU



THE PROBLEM



Air-Gapped Environments

Data centers prohibit cloud connectivity in server rooms — no API calls possible



Slow Manual Inspection

Technicians walk aisles checking thousands of LEDs, ports, and cables by eye



Sensitive Infrastructure Data

Serial numbers, rack topology, and server status are classified information



No Connectivity

Even connected DCs have dead zones between rack rows — WiFi is unreliable

THE SOLUTION

Point your phone at any server rack.

AI identifies every component in real-time.

All processing happens on-device — zero cloud.

16 Component Classes

Compute trays, ports, LEDs, cables, fans, drives, PSUs, DPUs, and more

RAG Knowledge Overlay

Tap any detection → specs, troubleshooting, LED meanings, maintenance steps

CPU vs NPU Toggle

Live comparison: XNNPACK (CPU)
↔ QNN HTP (Snapdragon NPU) with FPS/latency

WHY ON-DEVICE?

0ms

Cloud Latency

No round-trip — inference directly on Hexagon NPU

100%

Offline

Works in air-gapped rooms with zero connectivity

**0
bytes**
Data Leaked

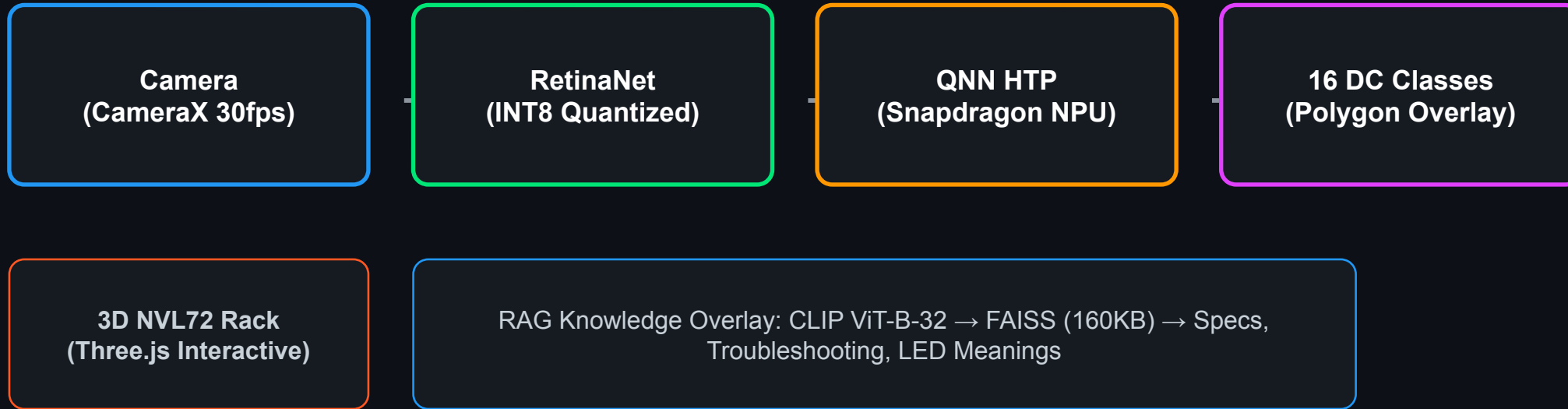
Serial numbers and infra data never leave the device

4-5x

Power Efficient

NPU vs CPU — longer battery life for extended inspections

ARCHITECTURE



Android App → ExecuTorch Runtime → QAIRT SDK → Snapdragon NPU

TECH STACK

Detection (NPU)	RetinaNet-ResNet50-FPN	QNN HTP, INT8, 36MB
Detection (CPU)	YOLOv8n-seg v3	XNNPACK, FP32, 13MB
RAG Retrieval	CLIP ViT-B-32 + FAISS	160KB index, on-device
Runtime	ExecuTorch 1.4	QNN backend for SM8750
App	Kotlin + CameraX	ExecuTorch AAR (3.4MB)
Device	Galaxy S25 Ultra	Snapdragon 8 Elite, Hexagon v79
3D Visualization	Three.js NVL72 Model	Interactive rack explorer

TRAINING PIPELINE

2,036 human-labeled images from 4 Roboflow datasets (CC BY 4.0)

Server Vision 1,293 imgs 30 classes (CPU, DIMM, fans, PSUs, drives)



PC Ports 327 imgs 4 classes (HDMI, RJ45, USB-A, USB-C)



Ports (ym) 138 imgs 7 classes (DP, ethernet, power, USB)



Server Detection 54 imgs 9 classes (SFP, XFP, patch panels)



50+ source classes → mapped to 16 DC-Ops classes

Augmentations: Moiré patterns, screen glare, brightness variation, color tint, scanlines

MODEL PERFORMANCE

0.749

mAP50 (box)

YOLOv8n-seg v3

0.85

Final Loss

RetinaNet (moiré aug)

6.4ms

Inference

Per frame on T4 GPU

compute tray0.924

network port0.912

power shelf0.832

LED indicator0.808

server rack0.683

drive bay0.912

DPU0.808

cooling manifold0.833

LIVE DEMO

0:00

Point phone at server rack / mini PC

Real-time colored polygon overlays appear

1:00

Tap detected component

RAG info panel: specs, troubleshooting, LED meanings

2:00

Toggle CPU ↔ NPU

Live FPS/latency comparison on screen

3:00

Enable airplane mode

Everything still works — fully offline

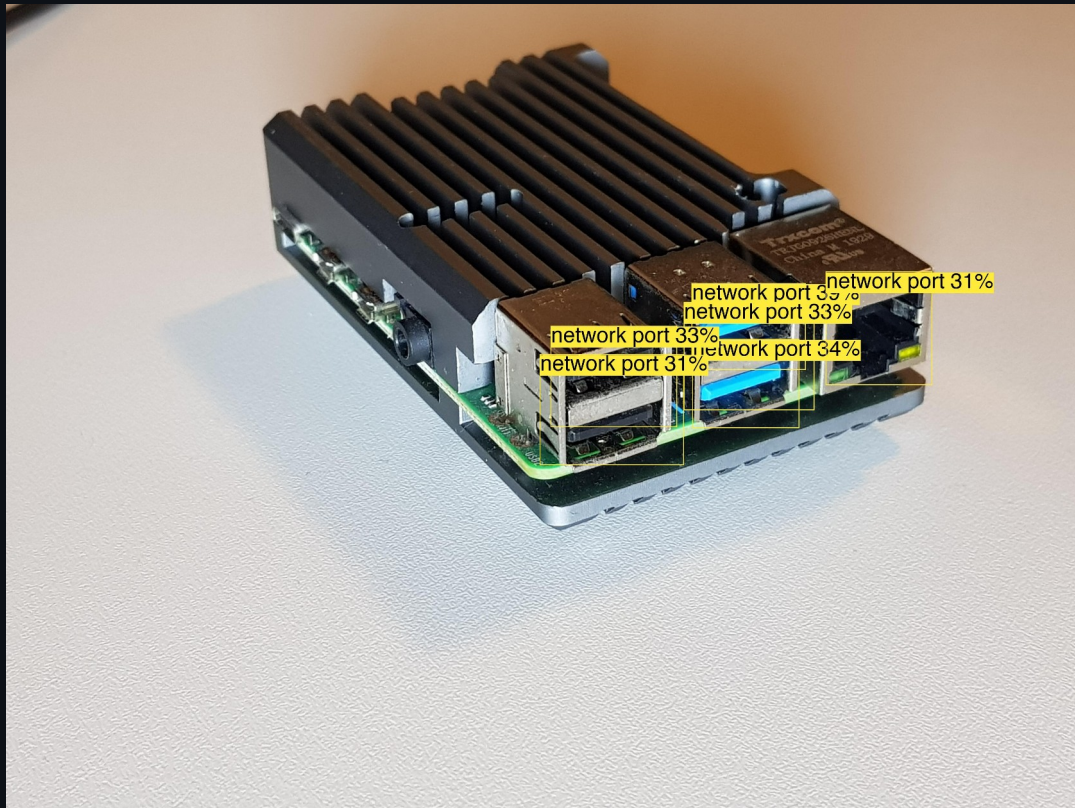
4:00

3D rack explorer

Interactive NVL72 model with component details

RETINANET IN ACTION

Real detections on a Raspberry Pi & USB hub — mirrors our physical demo prop



All USB + RJ45 ports detected as 'network port' — fully on-device

RAG KNOWLEDGE OVERLAY

Detect → Embed → Retrieve → Display

COMPUTE TRAY

92%

⚙️ 2x Grace CPU, 4x B200 GPU, 192GB HBM3e



Green LED = healthy and operational



Amber LED = check BMC log, verify coolant



Red LED = critical hardware failure



Maintenance: check cold plate contact,
verify NVLink connector seating

- ▶ 80 knowledge chunks
- ▶ CLIP ViT-B-32 embeddings
- ▶ FAISS vector index (160KB)
- ▶ NVL72-specific documentation
- ▶ All on-device, zero cloud

CPU vs NPU COMPARISON

Backend	XNNPACK (CPU)	QNN HTP (NPU)
Model	YOLOv8n-seg	RetinaNet
Size	13 MB	36 MB
Quantization	FP32	INT8
Est. Latency	~15-30ms	~3-5ms
Est. FPS	~30-60	~100+
Power	Higher drain	4-5x efficient
Hardware	ARM Kryo cores	Hexagon HTP v79

WHAT'S NEXT

Fine-grained Port Classification

Split 'network port' into HDMI, USB-A, USB-C, RJ45, SFP individually

LED State Detection

Classify LED colors (green/amber/red) and blink patterns for health scoring

On-Device LLM Integration

LLaMA 3.2 1B via ExecuTorch for natural language troubleshooting Q&A

Audit Log + Reporting

SQLite database of all inspections, exportable via secure channel

Multi-Rack Mapping

Combine depth estimation + detection for spatial rack layout understanding

DC-OPS

On-Device Data Center Operations Assistant

GitHub

`github.com/abhijitbetigeri/DC-Ops`

HuggingFace

`huggingface.co/datasets/abhijitbetigeri/dc-ops-dataset`

Models

`RetinaNet QNN HTP (.pte) + YOLOv8n-seg (.pte) + RAG index`

Built with ExecuTorch + Qualcomm QNN HTP + Snapdragon 8 Elite

Thank you!